

- The assignment is due at Gradescope on 4/18/25.
- A LaTeX template will be provided for each homework. You are strongly encouraged to type your homework into this template using $\text{\LaTeX}$. If you are writing by hand, please fill in the solutions in this template, inserting additional sheets as necessary. This will help facilitate the grading.
- You are permitted to discuss the problems with up to 2 other students in the class (per problem); however, *you must write up your own solutions, in your own words*. Do not submit anything you cannot explain. If you do collaborate with any of the other students on any problem, please list all your collaborators in the appropriate spaces.
- Similarly, please list any other source you have used for each problem, including other textbooks or websites.
- *Show your work*. Answers without justification will be given little credit.
- Your homework is *resubmittable*. Please refer to the course syllabus on Canvas for a more detailed description of this. For any problem that you have not changed from your last submission, please make sure to indicate this in your submission to help our graders grade faster.
- Each student can request a total of up to 5 days of extensions on all homework - either for original submission or resubmission. Up to two days can be requested for a homework. If you would like to request an extension, please fill out the [Homework extension form](#). Extensions do not carry any late penalty. If there is a life emergency that might prevent you from submitting on time not covered by this policy, email the Head TA.
- Is anyone still reading these?

PROBLEM 1 (LINEAR ALGEBRA PRACTICE) *This question is on the basics of linear algebra, please write out all steps for computation and formal proofs:*

(a) Let $A = \begin{pmatrix} 2 & 5 \\ 1 & 3 \end{pmatrix}$, compute the following:

(i) A^{-1}

(ii) A^T

(iii) A^2

(b) Let A and B be $n \times n$ matrices. What's wrong with the equation $(A + B)^2 = A^2 + 2AB + B^2$? Prove that the above equation holds if and only if A and B commute.

(c) Let A be a $m \times n$ matrix. Consider function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ where $f(x) = Ax$. Formally prove that f is a linear map (i.e. you need to show that $f(ax + by) = af(x) + bf(y)$ for all $x, y \in \mathbb{R}^n$ and $a, b \in \mathbb{R}$).

Collaborators:

Solution: Your solution here!

PROBLEM 2 (MODULAR FFT) *This question is based on problem 2.30 from [DPV].*

This problem illustrates how to do the Fourier Transform (FT) in modular arithmetic, for example, modulo 7.

- (a) *There is at least one number ω such that all the powers $\omega, \omega^2, \dots, \omega^6$ are distinct (modulo 7). Find one such ω , and show that $\omega + \omega^2 + \dots + \omega^6 \equiv 0 \pmod{7}$. (Interestingly, for any prime modulus there is such a number.)*
- (b) *Using the matrix form of the FT, produce the transform of the sequence $(0, 1, 1, 1, 5, 2)$ modulo 7; that is, multiply this vector by the 6×6 matrix $M_6(\omega)$, for the value of ω you found earlier. In the matrix multiplication, all calculations should be performed modulo 7.*
- (c) *Write down the matrix necessary to perform the inverse FT. Recall that dividing by a number amounts to multiplying by its inverse. Show that multiplying by this matrix returns the original sequence. (Again all arithmetic should be performed modulo 7.)*
- (d) *Now show how to multiply the polynomials $x^2 + x + 1$ and $x^3 + 2x - 1$ using the FT modulo 7.*

Collaborators:

Solution: Your solution here!

PROBLEM 3 (VERIFYING BINARY SEARCH TREES) For this problem you may assume that a binary tree node is defined as follows:

```
class node:
    left: node = None
    right: node = None
    value
```

You can assume that the values are distinct and the value type supports comparison. A binary tree T is called a binary search tree (BST) if and only if **for every node V in the tree**

$$\max_{U \in V.\text{left}} U.\text{val} < V.\text{val} < \min_{W \in V.\text{right}} W.\text{val}$$

- (a) Write pseudocode for a **recursive** algorithm that takes as input the root T of a binary tree and returns whether it is a BST. Your algorithm can optionally return more than just a boolean value. You are allowed to define helper functions if you want. Your algorithm should run in $O(n)$ time, where n is the number of nodes in tree T .
- (b) Prove the correctness of your algorithm.

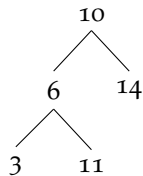
The height of a tree is defined as

$$\text{height}(T) = \begin{cases} 0 & T = \text{None} \\ 1 + \max(\text{height}(T.\text{left}), \text{height}(T.\text{right})) & \text{o.w.} \end{cases}$$

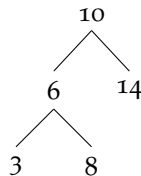
We would like all our Binary Search Trees operations (look-up, insertion, deletion) to have $O(\log n)$ time complexity. To achieve that, we require the BST to also be **balanced**. A BST is balanced if and only if **for every node V in the tree**

$$|\text{height}(V.\text{left}) - \text{height}(V.\text{right})| \leq 1$$

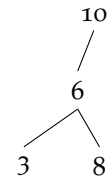
- (c) Modify below your pseudocode from part (a) to check if the binary tree T is a **balanced** BST. Your modified algorithm should also have $O(n)$ time complexity.



(a) This is not a BST since $11 > 10$



(b) This is a balanced BST



(c) This is a BST, but not balanced

Collaborators:

Solution: Your solution here!

PROBLEM 4 (DIVIDE-AND-CONQUER WITH ROOTS) In this question, we will find a divide-and-conquer algorithm for the following problem.

Input: A list $[r_1, r_2, \dots, r_n]$ of n numbers representing the roots of a polynomial $P(x)$. Thus,

$$P(x) = (x - r_1) \cdot (x - r_2) \cdot \dots \cdot (x - r_n)$$

Output: A list $[c_0, c_1, \dots, c_n]$ of coefficients for the same polynomial $P(x)$, so that

$$P(x) = c_0 + c_1 \cdot x + \dots + c_n \cdot x^n$$

For example, if $n = 2$ and we are given $[2, 1]$, then we would output $[2, -3, 1]$ as $(x - 2)(x - 1) = 2 - 3x + x^2$.

- (a) Provide pseudocode to solve this problem as efficiently as possible. You may assume that n is a power of 2. You may also use the function $\text{FFT}([a_0, \dots, a_n], \omega)$ which runs FFT given the coefficients $[a_0, \dots, a_n]$ of a degree- n polynomial, and the n -th root of unity ω .

Hint: a good starting point is to understand the FFT handout on Canvas. Remember that the function FFT is used to do both Evaluation and Interpolation step (with appropriate input).

Note, solutions which run in $\Omega(n^2)$ time will not receive credit.

- (b) Write a recurrence relation $T(n)$ for the running time of your algorithm in the box below. You do not need to justify your answer.

$T(n) =$

- (c) Solve the recurrence relation, and find the running time of your algorithm. **Show your work** and write your final running time using big-O notation in the box below.

$T(n) =$

Collaborators:

Solution: Your solution here!